

PYTHON PROGRAMMING INTERNSHIP

PROJECT 2: EXPENSE TRACKER

Complete Development Report

Powered by DecodeLabs
Batch: 2026

Prepared by: Muheeb-ur-Rehman
Role: Python Developer Intern

1. Introduction and Project Overview

Welcome to Project 2 of the DecodeLabs Python Programming Internship, Batch 2026. In Project 1, you mastered Data Management using lists. Now you step into the role of a Python Developer for the processing phase: The Expense Tracker.

This project is not about 'simple arithmetic.' It is about Data Accumulation — learning how to store, update, and process numerical data in real-time. Before building complex financial software, you must master the pattern that powers every ledger, bank account, and accounting system in the world: the accumulator pattern.

Mentor's Note: The training material literally calls this 'The Architecture of Financial Truth.' Treat every line of this project as if real money depended on its correctness — because in a production system, it would.

1.1 Project Objective

Build a Python script where:

- The user repeatedly enters expense amounts (e.g., 100, 50, 20)
- The program adds each amount to a running total
- The program displays the Total Spent once the user is finished entering expenses

By completing this milestone, you prove you can build a script that handles continuous data entry through pure mathematical and programmatic logic.

1.2 Problem Statement

People spend money in small, scattered transactions throughout the day — coffee, groceries, transport — and rarely track the cumulative effect. This program solves that problem by capturing each expense as it happens and maintaining a continuously updated total, giving the user an accurate, real-time picture of their spending.

1.3 Why This Project Matters

According to the official training material, this is 'the vital bridge to professional development for every intern.' The core skill — the Accumulator Pattern ($\text{total} = \text{total} + \text{new_expense}$) — is described as 'The Heartbeat of the Ledger.' This exact pattern, generalized as $\text{State}(\text{new}) = \text{State}(\text{old}) + \text{Input}$, is the foundation of bank account balances, shopping cart totals, game scores, inventory counts, and analytics dashboards.

1.4 Qualification Criteria

Per the official internship guidelines:

- Requirement: Complete Project 2 to unlock other projects for the following week
- Standard: All projects must be verified for quality
- Mandatory: Project 2 is the vital bridge to professional development for every intern

1.5 Tools and Environment Required

Tool / Requirement	Details
Programming Language	Python 3.8 or higher
Code Editor	VS Code, PyCharm, or IDLE (any is fine)
Operating System	Windows, macOS, or Linux — all work identically
External Libraries	None required — only Python's built-in features
Terminal / Command Prompt	Required to run the program (python expense_tracker.py)
Git and GitHub Account	Required for final submission

2. Python Concepts You Must Understand

Before writing a single line of code, you must understand the building blocks. Below is every concept used in this project, explained in plain, beginner-friendly language with real-world analogies and runnable examples, mapped directly to the official training material's terminology.

2.1 Variables for Numeric State

A variable can hold a number just as easily as text. In this project, one special variable will hold the 'state' of our program — the running total of money spent.

```
total = 0 # the starting state: zero money spent so far
```

2.2 The Shift: From Storage to Computation

Your training material draws an important distinction: Project 1 was about STORAGE (Static) — simply holding data in a list. Project 2 is about COMPUTATION (Dynamic) — actively transforming data through a pipeline of Input, Validation, Transformation, and Output Reality.

'From holding data to managing a Work-in-Progress financial reality.' This is the conceptual leap from Project 1 to Project 2.

2.3 The IPO Model (System Blueprint)

Every backend engine follows this model, and your training material visualizes it as a full industrial assembly line:

- **INPUT: THE GATE** — Raw, unstructured data enters the system (the user typing a number)
- **PROCESS: THE ENGINE** — Validation and Transformation occur here (checking the input is a real number, then adding it to the total)
- **OUTPUT: THE DISPLAY** — Refined, actionable reality is presented (the final total shown to the user)

GARBAGE IN = GARBAGE OUT — this principle, stated explicitly in your training material,

is why input validation (Phase 1: The Gatekeeper) is not optional.

2.4 Type Conversion with int() and float() — The Gatekeeper

Whatever the user types using input() is always TEXT (a string) — even if it looks like a number. If you try to do math directly on text, Python does not calculate; it concatenates (joins) the text together, producing nonsense.

```
# THE DISASTER (string concatenation, not addition):
a = "100"
b = "50"
print(a + b)          # Output: '10050'  <-- WRONG! This is text-joining, not math

# THE TRUTH (type conversion first):
a = int("100")
b = int("50")
print(a + b)          # Output: 150  <-- CORRECT! This is mathematical addition
```

int() converts text into a whole number. Since expenses can include decimals (e.g., 4.50 for coffee), we will use float() instead, which converts text into a decimal number.

```
price_text = "4.50"
price_number = float(price_text)
print(price_number)    # Output: 4.5
print(price_number + 1) # Output: 5.5 (real math now works)
```

2.5 Defensive Coding: try / except — The Digital Poka-Yoke

Your training material calls this 'The Digital Poka-Yoke' — a manufacturing term for an error-proofing mechanism that makes mistakes physically impossible. In code, try/except is our error-proofing shield.

If a user types 'ten' instead of '10', calling float('ten') crashes the program with a ValueError. We use try/except to catch this crash gracefully instead of letting the whole program die.

```
try:
    expense = float(input("Enter expense amount: "))
    # if this line succeeds, the code below runs
except ValueError:
    # if conversion fails, THIS code runs instead of crashing
    print("Invalid input. Please enter a number.")
```

DEFENSIVE CODING = PREVENTATIVE MEASURES. This is a direct quote from your training material's Phase 1 slide. Always assume the user WILL eventually type something unexpected.

2.6 The Accumulator Pattern — The Heartbeat of the Ledger

This is the single most important concept in Project 2. An accumulator is a variable that keeps a running total by repeatedly adding new values to itself.

```
total = 0          # Step A: Initialization (start at zero)
total = total + 5   # total is now 5
```

```
total = total + 10    # total is now 15
total = total + 20    # total is now 35

# Pythonic shorthand using += :
total += 100         # total is now 135
```

Your training material expresses this as a universal formula: $\text{State}(\text{new}) = \text{State}(\text{old}) + \text{Input}$. This single idea is used everywhere: a bank balance after a deposit, a shopping cart total after adding an item, a game score after earning points, or a step counter on a fitness tracker.

2.7 The Anatomy of State: Initialization vs. Iteration

Your training material illustrates two critical zones using a conveyor belt diagram:

Zone	What Happens	Code Location
A. Initialization (Memory)	<code>total = 0</code> runs ONCE, before any loop begins. This 'deposit chute' establishes the starting state that will accumulate value over time.	Written BEFORE the while loop
B. Iteration (Amnesia)	If <code>total = 0</code> is mistakenly placed INSIDE the loop, it resets to zero on every single pass — a 'reset trigger' that discards all previously accumulated value into a 'trash pit'.	Must NOT be written inside the loop

CRITICAL BUG WARNING: If you place `total = 0` inside your while loop instead of before it, your accumulator will reset to zero every single time the loop repeats, and your total will never grow beyond the last entered expense. This is the single most common bug in this project.

2.8 while True: — The Logic Skeleton (Continuous Audit Loop)

Your training material calls this 'The Continuous Audit' — a loop that keeps requesting new transaction input and validating state indefinitely, until explicitly told to stop.

```
while True:
    # this code repeats forever...
    # ...until something inside explicitly calls break
```

2.9 The Kill Switch — Sentinel Values

A sentinel value is a special signal value that tells the program 'stop now' — distinct from normal data. Your training material visualizes this as an emergency stop button with two paths:

- Standard Path: Data (Money) arrives → gets Added to Total
- Sentinel Path: A special Command (e.g., the word 'quit' or 'done') arrives → triggers Break Loop

```
while True:
    user_input = input("Enter expense (or 'done' to finish): ")

    if user_input.lower() == "done":
        break # the sentinel value triggers a graceful shutdown
```

```
# otherwise, treat it as a normal expense and process it
```

2.10 Phase 3: Output — Decoupling Logic from Display

Your training material emphasizes separating the Model (the calculation logic, `total += expense`) from the View (the final formatted display, `FINAL TOTAL: $150.00`). This is the same Model-View separation introduced in Project 1, now applied to numerical processing instead of list display.

```
# MODEL (logic/computation):
total += expense

# VIEW (display/output) – separate, formatted presentation:
print(f"FINAL TOTAL: ${total:.2f}")
```

The `:.2f` inside the f-string is a format specifier meaning 'round to exactly 2 decimal places' — essential for displaying money correctly (e.g., `$150.00` instead of `$150.0` or `$149.999999999998`, which can occur due to floating-point precision quirks).

2.11 Functions for Modular Design

Just like Project 1, we separate our code into purpose-built functions, each handling one responsibility.

```
def get_expense():
    """Handles ONE responsibility: safely getting a valid expense from the user."""
    pass

def display_total(total):
    """Handles ONE responsibility: formatting and printing the final total."""
    pass
```

2.12 The Fractal Scale: You Are Building a Bank

Your training material makes a profound point: the exact same `total += expense` logic that runs in this small console script is, at its core, the same computational pattern used by real banks, blockchain ledgers, and distributed financial systems worldwide. The architecture scales from a single Python script to global infrastructure, but the underlying logic — `State(new) = State(old) + Input` — remains identical.

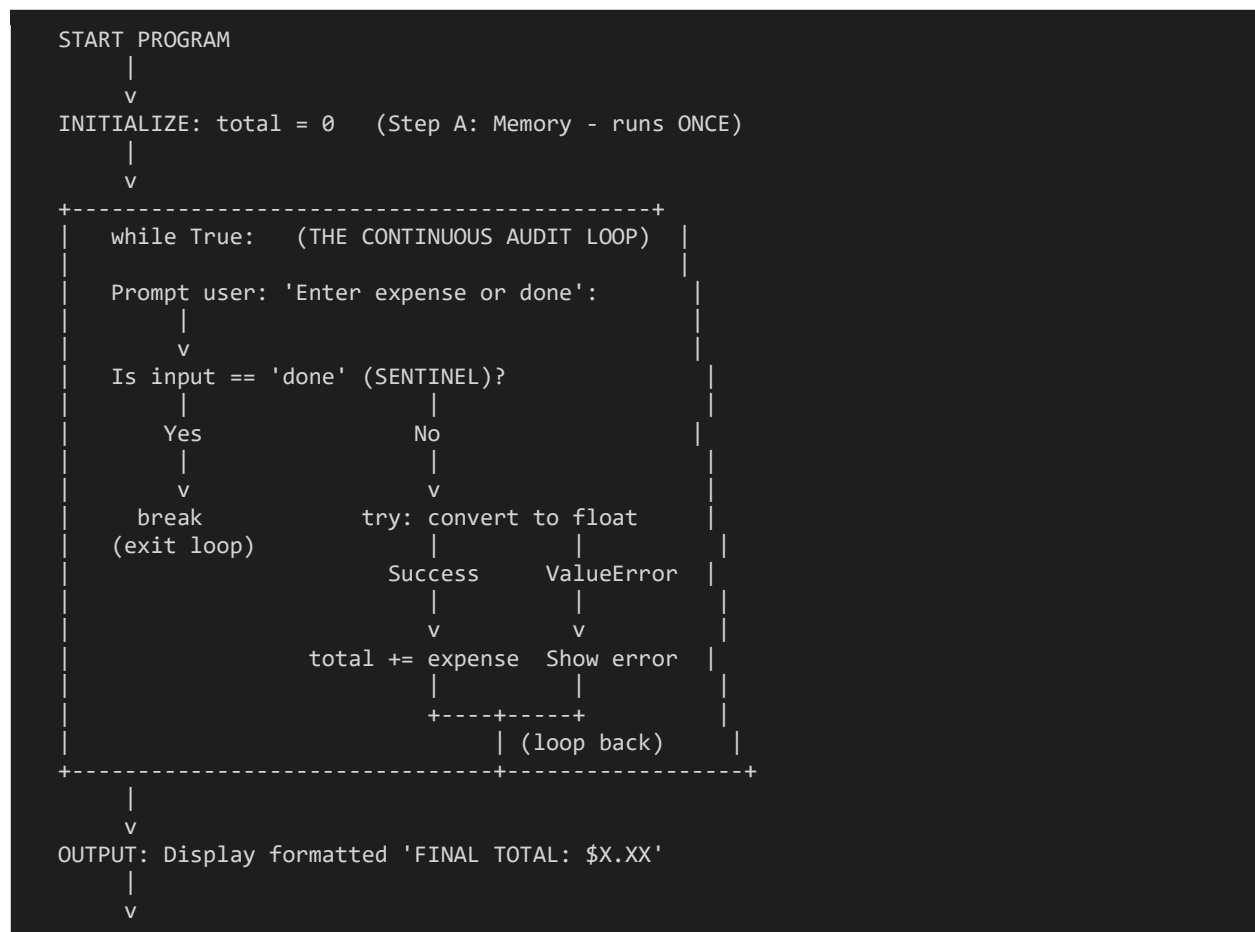
3. Project Planning and Roadmap

Professional developers always plan before coding. Below is your complete roadmap for Project 2, broken into manageable milestones.

3.1 Milestone Roadmap

1. Milestone 1: Set up your project file and initialize the accumulator (total = 0) OUTSIDE any loop
2. Milestone 2: Build the continuous input loop (while True) that requests expenses
3. Milestone 3: Implement the Sentinel Value (Kill Switch) so the user can stop entering expenses
4. Milestone 4: Implement Defensive Coding (try/except) to handle invalid, non-numeric input
5. Milestone 5: Implement the Accumulator Pattern (total += expense) for valid input
6. Milestone 6: Build the final Output / Display phase, showing the formatted Total Spent
7. Milestone 7: Add the if `__name__ == "__main__"`: professional entry point guard
8. Milestone 8: Test the program thoroughly, including edge cases and invalid input scenarios
9. Milestone 9: Review, document, and prepare for GitHub submission

3.2 Logical Workflow Diagram (Text Representation)



END PROGRAM

3.3 Inputs, Processes, and Outputs Breakdown

Stage	Description	Example
Input	Raw text the user types into the program	"100", "50.50", "done", or invalid text like "abc"
Process	Validation, type conversion, and accumulation	try/except converts text to float; total += expense adds it to the running sum
Output	The final, refined result shown to the user	"FINAL TOTAL: \$150.00"

3.4 What to Build First, Second, Third

1. FIRST: The state variable — total = 0, placed correctly outside any loop
2. SECOND: The loop skeleton — while True: with a basic prompt (no logic yet)
3. THIRD: The kill switch — sentinel value check and break, so the loop can actually end
4. FOURTH: The gatekeeper — try/except type conversion for defensive coding
5. FIFTH: The accumulator — total += expense, only reached when input is valid
6. SIXTH: The output — formatted final display after the loop ends
7. SEVENTH: The entry point guard — if __name__ == "__main__": main()

4. Task 2: Build the Expense Tracker — Step-by-Step Development

This is the heart of the document. Each step below builds on the previous one. Type each step into your `expense_tracker.py` file in the order given, and run the program after each major step to see how it behaves.

4.1 Step 1: Create Your Project File

Open your code editor and create a new file. Save it with this exact name:

```
expense_tracker.py
```

1. Open VS Code
2. Click File > New File
3. Press Ctrl+S (or Cmd+S on Mac) to save
4. Name the file `expense_tracker.py` and choose your project folder

4.2 Step 2: Add the File Header (Documentation Comment)

Professional code always starts with a comment block describing what the file does. Type this at the very top of your file:

```
"""
=====
PROJECT 2: EXPENSE TRACKER
=====
Organization : DecodeLabs Industrial Training
Batch       : 2026
Developer   : Faraz
Description  : A command-line Expense Tracker application
               that accumulates user-entered expenses in
               real-time using the Accumulator Pattern, and
               displays the total amount spent.
=====
"""
```

4.3 Step 3: Create the `get_expense()` Function (The Gatekeeper)

This function handles ONE job: safely retrieving a single piece of input from the user and converting it into either a valid number or a special signal.

```
def get_expense():
    """
    Prompt the user for an expense amount.
    Returns:
        float -> if the user entered a valid number
        None  -> if the user wants to quit (sentinel value)
        False -> if the user entered invalid (non-numeric) data
    """
    raw_input = input("\n Enter expense amount (or type 'done' to finish): ").strip()

    # Check for the sentinel value first
    if raw_input.lower() == "done":
```

```

        return None

    # Defensive coding: attempt the type conversion safely
    try:
        expense = float(raw_input)

        if expense < 0:
            print(" ❌ Expense cannot be negative. Please try again.")
            return False

        return expense

    except ValueError:
        print(" ❌ Invalid input. Please enter a numeric value (e.g., 100 or 49.99).")
        return False

```

Line-by-line explanation:

Line	What It Does
<code>def get_expense():</code>	Defines a function with a single, clear responsibility: get one valid expense or a stop signal
<code>raw_input = input(...).strip()</code>	Captures what the user types and removes accidental extra spaces
<code>if raw_input.lower() == "done":</code>	Checks for the sentinel value, case-insensitively (accepts 'done', 'Done', 'DONE')
<code>return None</code>	A special return value signaling 'the user wants to stop'
<code>try: expense = float(raw_input)</code>	Attempts to convert the text into a decimal number — the 'Gatekeeper' transformation
<code>if expense < 0: ... return False</code>	Defensive check: rejects negative expenses as logically invalid
<code>except ValueError: ... return False</code>	Catches the crash that would occur if the text cannot be converted to a number

4.4 Step 4: Create the `display_total()` Function

This function handles the OUTPUT phase — formatting and presenting the final result, fully decoupled from the calculation logic.

```

def display_total(total, count):
    """Display the final formatted total and number of expenses recorded."""
    print("\n" + "="*40)
    print("    EXPENSE SUMMARY")
    print("="*40)

    if count == 0:
        print("  No expenses were recorded.\n")
        return

    print(f"  Transactions Recorded : {count}")

```

```
print(f"  FINAL TOTAL SPENT      : ${total:.2f}")
print("="*40 + "\n")
```

Line-by-line explanation:

Line	What It Does
<code>def display_total(total, count):</code>	Receives the final accumulated total and the count of valid transactions
<code>if count == 0: ... return</code>	Handles the edge case where the user quit without entering any expenses
<code>print(f"...{count}")</code>	Shows how many valid expenses were recorded
<code>print(f"...\${total:.2f}")</code>	Displays the total formatted to exactly 2 decimal places, like real currency

4.5 Step 5: Create the main() Function (The Engine)

This is the controller that ties everything together: initialization, the continuous loop, the accumulator, and the final output.

```
def main():
    """Main controller function – runs the expense tracking engine."""

    # --- INITIALIZATION (Step A: Memory) ---
    # This MUST be outside the loop, or the total resets every iteration!
    total = 0.0
    count = 0

    print("\n" + "="*40)
    print("  📌 Welcome to DecodeLabs")
    print("      Expense Tracker")
    print("      Python Project 2 | Batch 2026")
    print("="*40)

    # --- THE CONTINUOUS AUDIT LOOP ---
    while True:
        result = get_expense()

        if result is None:
            # Sentinel value triggered -> graceful shutdown
            break

        if result is False:
            # Invalid input -> loop back and ask again, total unchanged
            continue

        # --- THE ACCUMULATOR PATTERN ---
        total += result
        count += 1
        print(f"  ✅ Added ${result:.2f} | Running Total: ${total:.2f}")

    # --- OUTPUT PHASE ---
    display_total(total, count)
```

Line-by-line explanation:

Line	What It Does
<code>total = 0.0</code>	Initializes the accumulator ONCE, before the loop — the critical 'Memory' step
<code>count = 0</code>	A second accumulator tracking how many valid expenses were entered
<code>while True:</code>	Starts the Continuous Audit Loop — runs indefinitely until break
<code>result = get_expense()</code>	Delegates input-gathering and validation entirely to the gatekeeper function
<code>if result is None: break</code>	Checks for the sentinel signal and exits the loop gracefully
<code>if result is False: continue</code>	Skips the rest of this loop pass on invalid input, looping back to ask again
<code>total += result</code>	The Accumulator Pattern in action — adds the new valid expense to the running total
<code>count += 1</code>	Increments the transaction counter
<code>display_total(total, count)</code>	Called once, after the loop ends, to show the final formatted result

Why 'if result is None' and 'if result is False' instead of just 'if result'? Because 0 (zero) is a perfectly valid expense in Python's eyes, but 'if result:' would treat 0 as falsy and incorrectly skip it. Using 'is None' and 'is False' makes our checks precise and bug-free.

4.6 Step 6: Add the Entry Point Guard

At the very bottom of your file, add this final piece:

```
if __name__ == "__main__":
    main()
```

4.7 Complete Final Source Code (expense_tracker.py)

Below is the entire file, combined. Verify your file matches this exactly before running it.

```
"""
=====
PROJECT 2: EXPENSE TRACKER
=====
Organization : DecodeLabs Industrial Training
Batch       : 2026
Developer   : Faraz
Description  : A command-line Expense Tracker application
               that accumulates user-entered expenses in
               real-time using the Accumulator Pattern, and
               displays the total amount spent.
=====
"""
```

```

"""

def get_expense():
    """
    Prompt the user for an expense amount.
    Returns:
        float -> if the user entered a valid number
        None -> if the user wants to quit (sentinel value)
        False -> if the user entered invalid (non-numeric) data
    """
    raw_input = input("\n Enter expense amount (or type 'done' to finish): ").strip()

    if raw_input.lower() == "done":
        return None

    try:
        expense = float(raw_input)

        if expense < 0:
            print(" ☐☐ Expense cannot be negative. Please try again.")
            return False

        return expense

    except ValueError:
        print(" ✕Invalid input. Please enter a numeric value (e.g., 100 or 49.99).")
        return False


def display_total(total, count):
    """Display the final formatted total and number of expenses recorded."""
    print("\n" + "="*40)
    print(" ☐ EXPENSE SUMMARY")
    print("="*40)

    if count == 0:
        print(" No expenses were recorded.\n")
        return

    print(f" Transactions Recorded : {count}")
    print(f" FINAL TOTAL SPENT : ${total:.2f}")
    print("="*40 + "\n")


def main():
    """Main controller function – runs the expense tracking engine."""

    total = 0.0
    count = 0

    print("\n" + "="*40)
    print(" ☐ Welcome to DecodeLabs")
    print(" Expense Tracker")
    print(" Python Project 2 | Batch 2026")
    print("="*40)

```

```
while True:
    result = get_expense()

    if result is None:
        break

    if result is False:
        continue

    total += result
    count += 1
    print(f" ☒ Added ${result:.2f} | Running Total: ${total:.2f}")

display_total(total, count)

if __name__ == "__main__":
    main()
```

4.8 How to Run the Program

1. Open your terminal / command prompt
2. Navigate to the folder where `expense_tracker.py` is saved using `cd`
3. Type the command: `python expense_tracker.py` (or `python3 expense_tracker.py` on some systems)
4. Press Enter — your program should now display the welcome message and prompt for an expense

5. Testing the Program

Testing is not optional. A professional developer always tests every possible path through their program before considering it complete.

5.1 Sample Test Run (Full Walkthrough)

```
=====
❏ Welcome to DecodeLabs
  Expense Tracker
  Python Project 2 | Batch 2026
=====

Enter expense amount (or type 'done' to finish): 100
✅Added $100.00 | Running Total: $100.00

Enter expense amount (or type 'done' to finish): 50
✅Added $50.00 | Running Total: $150.00

Enter expense amount (or type 'done' to finish): 20
✅Added $20.00 | Running Total: $170.00

Enter expense amount (or type 'done' to finish): done

=====
❏ EXPENSE SUMMARY
=====
Transactions Recorded : 3
FINAL TOTAL SPENT    : $170.00
=====
```

5.2 Test Cases Table

#	Test Scenario	Input	Expected Output
1	Add a valid integer expense	"100"	Accepted; total increases by \$100.00
2	Add a valid decimal expense	"49.99"	Accepted; total increases by \$49.99
3	Add invalid (non-numeric) text	"ten"	Error message shown; total unchanged; loop continues
4	Add a negative number	"-20"	Rejected with warning; total unchanged; loop continues
5	Quit immediately without entering any expense	"done" (first input)	Summary shows 'No expenses were recorded.'
6	Quit after several valid entries	100, 50, 20, then "done"	Correct cumulative total displayed: \$170.00
7	Sentinel value case-	"DONE" or "Done"	Recognized as the quit signal

	insensitivity		regardless of capitalization
8	Expense with extra spaces	" 75 "	Accepted as 75.0 after <code>.strip()</code> removes spaces
9	Empty input (just Enter key)	"" (blank)	Caught by except <code>ValueError</code> , since <code>float("")</code> raises an error
10	Many transactions (10+)	Repeatedly entering numbers	Accumulator correctly sums all entries without resetting

5.3 Edge Cases to Consider

- What if the user enters '0' as an expense? It is mathematically valid and should be accepted (0 is not negative, and is a legitimate numeric value)
- What if the user enters a very large number (e.g., 999999999)? Python's float type handles large numbers without special handling needed for this project's scope
- What if the user enters a number with many decimal places (e.g., 19.9999999)? The `:.2f` formatting in `display_total()` rounds it correctly for final display
- What if the user mixes valid and invalid entries? Each entry is evaluated independently; invalid ones are rejected without affecting the accumulated total
- What happens if the program is closed without typing 'done' (e.g., Ctrl+C)? The total is lost since it only exists in RAM — expected behavior for this version, to be solved with persistence in advanced versions

6. Debugging

Even experienced developers make mistakes. Below are the most common errors for this project and exactly how to fix them.

6.1 Error: total Resets to Zero Every Loop

Cause: `total = 0` was mistakenly placed **INSIDE** the while loop instead of before it. Your training material calls this 'Iteration (Amnesia)' — the accumulator's value gets discarded into the 'trash pit' on every pass.

```
# WRONG – resets every loop, total never grows past the last entry
while True:
    total = 0
    expense = float(input("Enter expense: "))
    total += expense

# CORRECT – initialized once, accumulates correctly
total = 0
while True:
    expense = float(input("Enter expense: "))
    total += expense
```

6.2 Error: ValueError: could not convert string to float

Cause: The user typed something that is not a valid number (e.g., 'ten', 'abc', or left it blank), and there was no try/except to catch the failure.

```
# WRONG – crashes the entire program on bad input
expense = float(input("Enter expense: "))

# CORRECT – catches the error gracefully
try:
    expense = float(input("Enter expense: "))
except ValueError:
    print("Invalid input. Please enter a number.")
```

6.3 Error: String Concatenation Instead of Addition

Cause: Forgetting to convert `input()` text into a number before adding it, resulting in text-joining instead of arithmetic.

```
# WRONG – '100' + '50' becomes '10050' (text joined, not added)
expense = input("Enter expense: ")
total = total + expense

# CORRECT – convert first, then add
expense = float(input("Enter expense: "))
total = total + expense
```

6.4 Error: Program Never Stops (Infinite Loop)

Cause: The sentinel value check is missing or written incorrectly, so 'break' is never reached.

```
# WRONG – no way to exit the loop
while True:
    expense = input("Enter expense: ")
    total += float(expense)

# CORRECT – sentinel value provides an exit path
while True:
    raw = input("Enter expense (or 'done'): ")
    if raw.lower() == "done":
        break
    total += float(raw)
```

6.5 Error: `TypeError: unsupported operand type(s)`

Cause: Trying to add a number to a string, or comparing incompatible types, often happens if conversion was applied inconsistently.

```
# WRONG – mixing a string and a float in addition
total = 0
expense = input("Enter expense: ") # this is a STRING
total = total + expense           # TypeError!

# CORRECT – ensure consistent numeric types throughout
total = 0.0
expense = float(input("Enter expense: "))
total = total + expense
```

6.6 General Debugging Techniques

1. Read the error message from the **BOTTOM** up — the last line tells you the actual error type and message
2. Note the line number mentioned in the error — Python tells you exactly where it failed
3. Use `print(total)` temporarily after each accumulation step to verify the running total is correct
4. Test with simple, predictable numbers first (e.g., 10, 10, 10 should total 30) before testing decimals or edge cases
5. If the total seems wrong, check **FIRST** whether `total = 0` is inside or outside your loop